Paritosh Mathur - 10th Aug 2026

Claim Assistant — Requirements

1. Overview

Claim Assistant is an agentic system that researches and adjudicates billing dispute or fraud claims on behalf of a human claim processor. Today, a processor manually gathers evidence from multiple systems (transaction history, account red flags, policy documents, regulations) before deciding to approve or deny a claim. Claim Assistant automates that research-and-decide loop: it reads a submitted claim, determines which checks apply to its type, autonomously gathers evidence via specialized tools/agents, and either renders a decision (Approve/Deny) or reports an inconclusive result with the reason evidence could not be completed.

This is a general-purpose "research → verify → decide" pattern applicable beyond claims to any domain requiring auditable, evidence-based decisions (underwriting, KYC review, compliance case review, etc.).

Primary user: Claim Processor — reviews agent-produced decisions, supplies missing information when asked, and remains the accountable human in the loop.

2. Problem Statement

Claims research is slow and inconsistent because it requires cross-referencing several independent sources (transaction history, account history, applicable policy/regulation text) and reasoning over incomplete information. A single LLM prompt cannot solve this:

- The correct research path depends on the **claim type**, discovered only after reading the claim.
- Each research step's result determines the **next** step — this is inherently iterative, not a one-shot completion.
- The agent must know when it has **enough** evidence to decide, and must recognize when it does **not** (and ask a human, rather than guess).
- Decisions must be **grounded and citable** (policy/regulation reference) because claims are subject to internal and external audit.

A stand-alone prompt has no mechanism for tool use, multi-step planning, evidence gathering, or a controlled stopping condition — Claim Assistant needs a reasoning loop with tools, memory, and explicit termination logic instead.

3. Environment

Component	Role
Claims Application	System of record. Source of the claim; destination for the final determination.
Claim Processor	Human user. Submits claims for review, answers agent questions, receives the decision.
Synthetic research sources	Simulated data sources for the capstone: customer transaction history, system/access logs, customer account data (fraud/red-flag signals).
Policy & regulation knowledge base (RAG)	Company policy and government regulation documents, retrieved via semantic search to ground and cite decisions.
Audit trail	Append-only log of every claim decision and the evidence/reasoning basis for it.

4. Agent Actions

The agent can take the following categories of action:

1. **Read** the claim submitted by the processor.
2. **Classify** the claim to determine which regulatory/compliance/internal-policy checks apply.
3. **Research**, using one or more tools/specialized agents, to gather the evidence each check requires. Which tool is called next depends on the outcome of the previous call.
4. **Ask a human** when a required piece of information cannot be obtained from any tool, then resume research once the answer is supplied.
5. **Decide**: after each step, evaluate whether all required checks have a verified outcome.
   - All checks pass → **Approve**
   - Any check fails → **Deny**
   - Required checks cannot be completed (missing information, tool/human budget exhausted) → **Inconclusive**, with a stated reason.

5. Reasoning Loop (ReAct-style)

The agent runs a **Think** → **Act** → **Observe** loop:

1. **Think** — Given short-term memory (this claim's accumulated findings) and a summary of long-term memory (policy corpus, known entity facts), decide the next

step. On iteration 1 this typically means identifying which regulations/internal procedures apply to this claim type. On later iterations it uses the more precise information returned by prior tool calls.

2. **Act** — Take one action against the environment, almost always via a tool: fetch more evidence (fraud score, related claims, transaction detail, regulation text), or ask a human a question. In the final step, the action is writing the claim decision back to the Claims Application.
3. **Observe** — Inspect the tool/human result together with current short- and long-term memory state.
4. **Terminate or loop** — If a final decision has not yet been reached, return to Think. A run terminates when:

    - Every required check has a verified **PASS** → Approve.
    - Any required check is a verified **FAIL** → Deny (short-circuits remaining checks).
    - A hard iteration cap is reached (design ceiling: ~12 iterations, or 5 iterations without measurable progress toward resolving a check) → Inconclusive.
    - The human-question budget is exhausted → Inconclusive.

    The decision is **derived from the state of the checks**, never self-declared by the model directly — this prevents the agent from asserting "Approve" without the underlying evidence actually supporting it.

Why ReAct rather than Tree of Thought

Tree of Thought (ToT) generates and evaluates multiple candidate reasoning branches in parallel and searches/backtracks over them — it fits problems where the *path* to a solution is uncertain and worth exploring multiple ways before committing (e.g. puzzle-solving, multi-step planning with no clear next move). Claim adjudication doesn't have that shape:

- **The next step is usually determined by what's still unresolved, not by a choice among competing strategies.** Each required check is either PASS, FAIL, UNKNOWN, or BLOCKED; the next action is "go resolve an UNKNOWN or BLOCKED check," not one of several plausible directions that need to be weighed against each other. There's rarely a real branch point to explore.
- **Evidence must accumulate linearly and auditably.** A regulator or internal auditor reviewing a decision needs a single, ordered trace of what was checked and why — "the agent tried three parallel lines of reasoning and picked the best one" is a materially harder thing to justify after the fact than "the agent worked through checks 1 through N in order, and here's what each one found." ReAct's

single linear trajectory maps directly onto the audit trail requirement (§11); ToT's branching/pruned paths would either bloat the audit trail with abandoned branches or require deciding which branches are even worth recording.
- **Termination is defined by exhausting a fixed set of checks, not by search convergence.** ToT's value comes from searching a space until a good-enough solution is found; this problem instead has a closed, enumerable set of required checks per claim type, so a single pass that resolves them one at a time (with a bounded iteration/human-question budget, §5 step 4) is a natural fit and cheaper than exploring parallel branches that mostly wouldn't apply.
- **Cost and latency scale with branching.** ToT's parallel-candidate generation multiplies LLM calls per step; for a workload already bounded by tool/API latency and a per-claim iteration cap, that multiplier isn't justified by a corresponding gain in decision quality here.

ReAct's linear Think → Act → Observe loop, paired with the check ledger as the external source of truth, was chosen because it matches the actual shape of the problem — sequential evidence-gathering against a known checklist with a clear stopping condition — rather than because ToT wasn't considered.

6. Decision Model

Each required check for a claim type has one of four states, tracked in a **check ledger**:

- **PASS** — verified true by a tool result or human-confirmed fact.
- **FAIL** — verified false; triggers an immediate Deny.
- **UNKNOWN** — not yet evaluated, or evidence so far is inconclusive.
- **BLOCKED** — cannot be evaluated (e.g., a dependent check failed, or a tool/human is unavailable).

Only tool-verified or human-verified facts may close (PASS/FAIL) a check. Agent-inferred facts (the model's own reasoning about the evidence) may only suggest the next action — they cannot themselves satisfy a check. This distinction exists to keep every Approve/Deny traceable to an external, checkable fact rather than to the model's own unverified inference.

7. Memory

Short-term memory (per-claim)

- Accumulates as the claim is researched: findings from each iteration, tool results, human answers, current check-ledger state.

- Must survive pauses: when the agent asks a human a question, the loop suspends and its state needs to be persisted (not just held in-process) so it can be rehydrated correctly when the human responds, possibly hours or days later.

Long-term memory (cross-claim)

- Persistent knowledge shared across all claims/sessions, not tied to one case.
- **Semantic memory** — the policy/regulation corpus (bulletins, procedures, letter templates), held in a versioned vector store and retrieved via similarity search.
- **Episodic memory** — facts about recurring entities (e.g., a given account or cardholder), tagged with their provenance (which claim/tool established the fact), so previously verified information doesn't need to be re-derived from scratch on the next claim touching the same entity.
- Critical facts obtained early in a run (e.g., the governing regulation identified in iteration 1) must not be silently dropped by summarization or context-window eviction — long-term storage exists specifically to protect against that failure mode.

8. Tools

Tool category	Purpose	Addresses
Retrieval	Claim/dispute history, policy/regulation search	Retrieval limitation — LLM has no access to current/private data
Grounding	Transaction evidence lookup, account/cardholder verification, document extraction with citations	Grounding limitation — claims must be tied to a checkable, citable fact
Computation	Date math, duplicate-charge detection, transaction-pattern anomaly scoring	Computation limitation — LLMs are unreliable at precise arithmetic
ask_human	Retrieval of last resort when no tool can supply required information	Retrieval limitation, human judgment gap
write_determination	Writes the final decision + basis back to the Claims DB	Single irreversible action — deliberately isolated as the only write

`write_determination` is treated as a single irreversible write and is only invoked once the check ledger supports a terminal decision — it is the one tool call in the loop that cannot be undone, so it is intentionally the last action in a run.

9. Retrieval-Augmented Generation (RAG) Design

Why retrieval is required: Policy and regulation text changes over time, decisions must cite the specific policy/regulation used, and those citations are subject to internal and external audit. The agent must always ground its reasoning in the current version of the applicable text rather than the model's training data.

Approach

- Source documents: policy/regulation Word/PDF documents (capstone); a more scalable, versioned document store is required for a production system.
- Chunking: segment by policy/regulation clause, not by fixed size, so a single provision (e.g., "maximum days allowed for investigation") is never split across chunks. Preserve parent-child relationships between a policy and its sub-clauses. Tables and decision trees are converted to markdown and kept whole within a single chunk.
- Embeddings + similarity search over a vector database (Pinecone).
- Retrieval pipeline: retrieve top **k ≈ 20** candidates → rerank → apply a **relevance-floor** threshold → return top **3**. If nothing clears the relevance floor, the system returns **zero results** — "no matching policy found" is treated as a valid, honest outcome rather than forcing a low-confidence match.
- Full retrieval detail (query, filters, all candidate results and scores, not just the top 3 returned) is logged to the audit trail, since the retrieval process itself may be reviewed by auditors later.

How retrieval changes the output: Policies that vary by organization — e.g., dollar thresholds for auto-approval, or which communication channel (letter/text/push/phone) is used for a given claim outcome — are only knowable via the retrieved policy context. Without retrieval, the agent would be reasoning from generic or stale knowledge instead of the organization's actual current rules, and could not attach a citation to its decision.

Known failure mode: Retrieving a policy that matches lexically but not by claim type (e.g., an ATM-transaction policy surfacing for a Check claim) could ground the decision in the wrong rule set. Mitigation: a deterministic rule-based check that the retrieved policy's claim-type tag matches the claim being processed, applied before the policy is allowed to close a check.

10. Human-in-the-Loop

- `ask_human` is the tool of last resort — used only when no available tool can supply a required fact.

- Invoking it **suspends** the run; state is serialized (short-term memory + check ledger) so the run can be resumed accurately whenever the processor responds.
- There is a bounded human-question budget per claim; exhausting it without resolving all required checks results in an Inconclusive outcome rather than an indefinite wait.

11. Audit & Compliance

- Every decision (Approve/Deny/Inconclusive) is written with its full evidentiary basis — which checks passed/failed/were blocked, and what evidence or citation supports each.
- The audit trail is append-only.
- Full retrieval activity (queries, filters, all candidates and scores) is captured, not just what was ultimately used, so the reasoning behind a decision can be reconstructed after the fact.

12. Evaluation Plan

- A seed set of **10 test claims**, spanning different claim types and levels of information completeness (including cases with deliberately missing information), each with a predetermined expected outcome.
- Agent outputs (decision + reasoning trace) are compared against the predetermined outcomes to validate correctness of both the final decision and the path taken to reach it.

13. Non-Functional Requirements

- **Traceability** — every decision must be reconstructable from the audit trail alone, without relying on model memory.
- **Resumability** — the orchestrator must survive process restarts/pauses while a claim is waiting on a human answer (async job, not an in-memory-only loop).
- **Determinism of decisioning** — the Approve/Deny/Inconclusive outcome is computed from the check ledger state via fixed rules, not left to free-form model output.
- **Boundedness** — iteration count and human-question count are both capped to guarantee every run terminates.